

Spark 原理详解

第 15 讲：DataFrame 与 Spark SQL

从“对象集合”进入“Spark 能理解和优化的结构化数据”

本讲位置：第 15 / 20 讲。完成后还剩 5 讲：Catalyst 与执行计划、Spark on YARN、性能优化、综合实战、MapReduce + Spark 总复习。

学习目标：学完后，你应该能解释 DataFrame 为什么比 RDD 更容易被 Spark 优化；看懂 Schema、Row、Column；使用 select/filter/withColumn/groupBy/agg/orderBy/join；理解 DataFrame API 与 SQL 的关系；并能判断哪些操作可能触发 Shuffle。

这讲的关键不是“学会几条 SQL”，而是理解：当 Spark 知道每一列叫什么、是什么类型、要做什么关系运算以后，它就有机会在真正执行前重写和优化你的计算计划。

1. 为什么有了 RDD，还要 DataFrame

RDD 给了程序员很强的自由度：一个元素可以是数字、字符串、元组，甚至任意自定义对象。但这种自由也有代价。对 Spark 来说，自定义 Python 函数内部的业务语义通常是不透明的。Spark 知道“这里要调用一个函数”，却未必知道这个函数是在过滤 city、选择 amount，还是进行完全不同的计算。

RDD 与 DataFrame：最大差别不是“数据长得像表格”

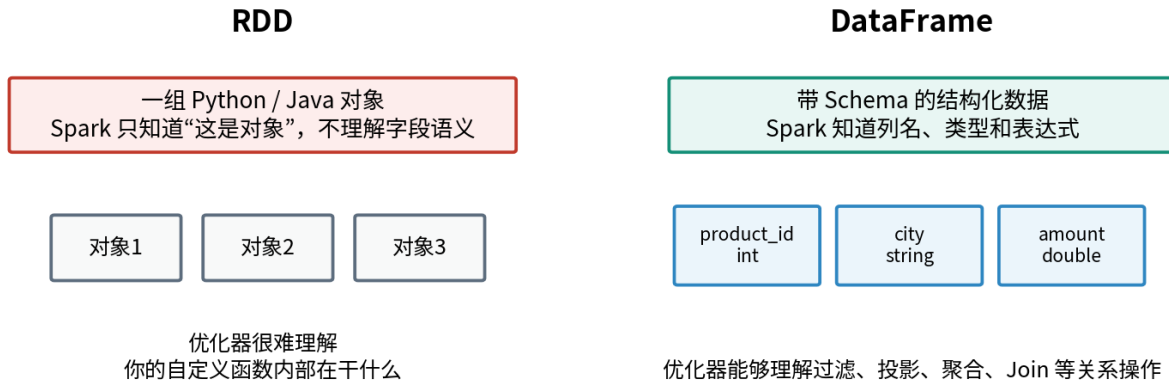


图 1 RDD 与 DataFrame：关键差别是 Spark 是否理解数据结构与表达式

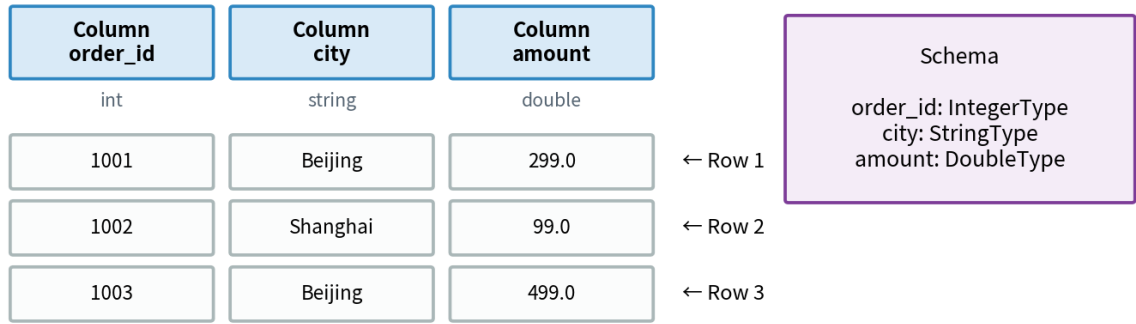
底层原因：DataFrame 把“数据”和“元数据”放在一起：列名、数据类型、表达式、聚合关系都变得显式。于是 Spark 可以在执行前做列裁剪、谓词下推、Join 策略选择等优化。具体优化机制将在第 16 讲 Catalyst 中展开。

所以，不应该把 DataFrame 只理解成“看起来像 Excel 的二维表”。更准确地说，它是 Spark SQL 引擎能够理解的一种结构化分布式数据抽象。

2. DataFrame 到底是什么

在概念上，可以把 DataFrame 理解为“带有命名列和 Schema 的分布式数据集”。在 PySpark 中，这是日常结构化数据处理最常用的抽象。它底层仍然会被切成 Partition，在 Executor 上由 Task 并行处理；并不会因为用了 DataFrame 就变成单机表格。

DataFrame 的四个基本概念：Schema、Row、Column、DataFrame



DataFrame = 多个 Row + 明确的 Schema；Column 是“列/表达式”的抽象，不只是一个字符串名字。

图2 DataFrame、Schema、Row 与 Column 的关系

最容易误解的点：DataFrame 不是“把所有数据装进 Driver 的二维数组”。它仍然是分布式的；Driver 更多保存查询计划和元数据，真正的数据通常分散在 Executor / 外部存储中。

3. Schema：为什么数据类型如此重要

Schema 可以理解为 DataFrame 的结构说明书。它告诉 Spark：有哪些列、每一列叫什么、是什么类型、是否允许 NULL。

```
root
|-- order_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- city: string (nullable = true)
|-- amount: double (nullable = true)
|-- status: string (nullable = true)
```

有了 Schema，Spark 才能正确理解很多操作。例如 `amount > 100` 是数值比较，而不是字符串字典序比较；`SUM(amount)` 是数值聚合；日期列可以使用日期函数。

典型错误：如果金额 "100" 和 "20" 被当成字符串，排序可能得到 "100" < "20" 这样的结果。类型不是装饰信息，它直接决定计算语义。

读取结构化数据：Schema 从哪里来？

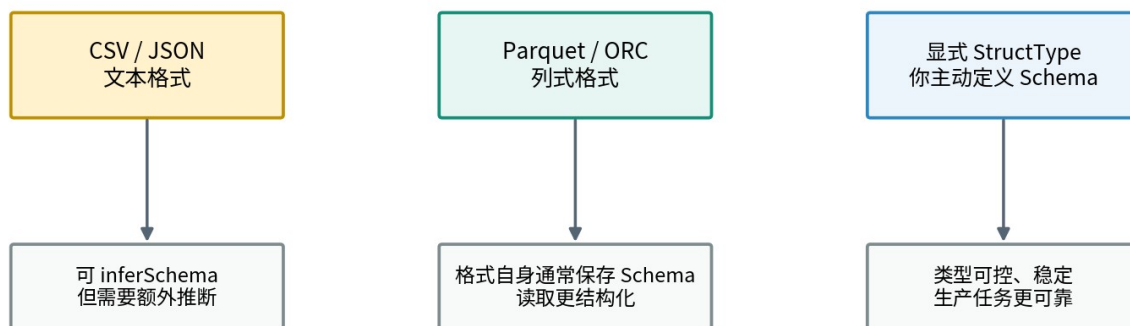


图 3 Schema 可以来自格式自身、类型推断或显式定义

学习阶段使用 `inferSchema` 很方便，但在稳定的生产任务里，显式 Schema 往往更可靠：类型不会因为某批样本异常而改变，也避免为了推断类型而额外扫描数据。

4. Row：一行数据不是普通 Python dict

DataFrame 的每一条逻辑记录可以用 Row 表示。Row 让一行数据按照列名或位置访问。你可以把它类比为数据库表的一行，但它属于 Spark 的结构化记录抽象。

```
rows = df.take(3)
print(rows[0])
# Row(order_id=1001, product_id=10, city='Beijing', amount=299.0)

print(rows[0]['city'])
# Beijing
```

注意：take()、collect() 会把结果发送到 Driver。小规模观察可以使用；面对大数据不要用 collect() 把整个 DataFrame 拉回本机。

5. Column：它不只是“列名字符串”

Column 是初学者最容易忽略、但非常重要的抽象。df["amount"]、col("amount")、df.amount 都可以产生 Column 表达式。之后的加减乘除、比较、条件判断，本质是在构造表达式树。

```
from pyspark.sql import functions as F

# 这些都在构造 Column / Expression
amount_col = F.col("amount")
large_order = F.col("amount") >= 300
tax_amount = F.col("amount") * F.lit(0.06)

result = df.select(
    "order_id",
    "amount",
    tax_amount.alias("tax")
)
```

这里真正有意思的地方是：Spark 能看到 “amount * 0.06” 这个表达式，而不是只看到一段任意 Python 代码。这样优化器就能参与。

科研/工程习惯：能用 Spark 内置函数表达的逻辑，优先使用内置函数和 Column 表达式。Python UDF 很方便，但会让部分语义重新变得不透明，并可能引入额外序列化边界。UDF 不是不能用，而是不要把它当默认第一选择。

6. 第一个完整 DataFrame 示例：销售数据

假设 HDFS 中有 sales.csv：

```
order_id,product_id,city,amount,status
1001,10,Beijing,299.0,paid
1002,11,Shanghai,99.0,cancelled
1003,10,Beijing,499.0,paid
1004,12,Guangzhou,199.0,paid
1005,11,Shanghai,399.0,paid
```

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("SalesDF").getOrCreate()

df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv("hdfs:///input/sales.csv"))

df.printSchema()
df.show()
```

这里的 read 并不意味着整个文件已经被完整计算完并搬到 Driver。DataFrame 仍然参与 Spark 的惰性执行与分布式读取。

7. select、filter、withColumn：最常见的三类 Transformation

可以把它们分别理解成：选列、选行、构造新列。

```
from pyspark.sql import functions as F

# 1. select: 只选择需要的列
small = df.select("product_id", "city", "amount")

# 2. filter / where: 只保留满足条件的行
paid = df.filter(F.col("status") == "paid")

# 3. withColumn: 构造或替换一列
with_tax = paid.withColumn(
    "amount_with_tax",
    F.col("amount") * 1.06
)
```

这些通常是 Transformation：它们主要向执行计划中添加节点。Spark 可以把多个连续操作组合起来，而不是机械地“每写一行就生成一个新的中间文件”。

DataFrame 仍然是惰性执行：先构建计划，再由 Action 触发执行

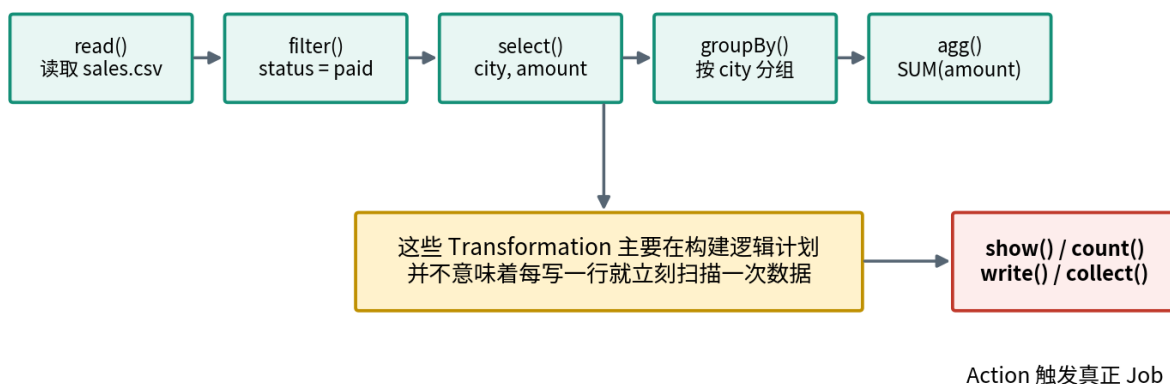


图 4 DataFrame 仍然遵循 Transformation → Action 的惰性执行模型

对照 RDD：RDD 的 map/filter 也是惰性的；DataFrame 的优势不是“才有惰性执行”，而是 Spark 还能理解 select/filter/groupBy 等高层关系语义。

8. groupBy + agg：从业务问题到结构化聚合

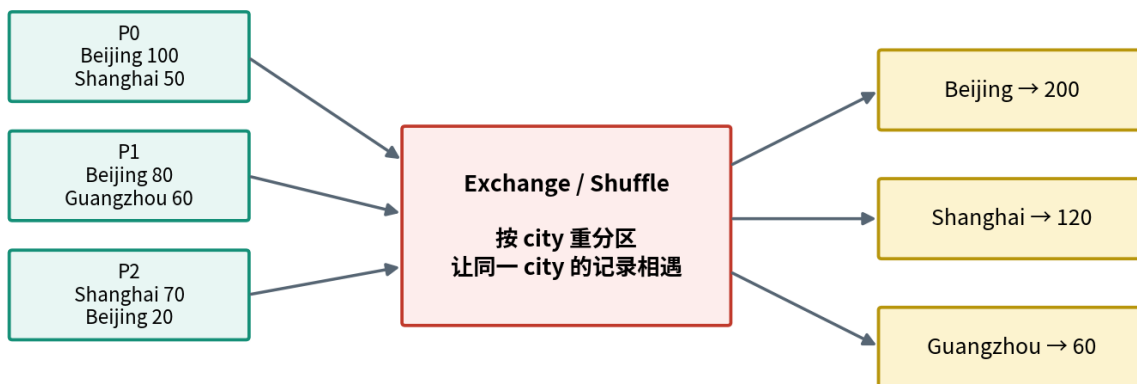
现在要求：只统计 paid 订单，并计算每个城市的销售额和订单数。

```
city_stats = (df
    .filter(F.col("status") == "paid")
    .groupBy("city")
    .agg(
        F.sum("amount").alias("sales"),
        F.count("*").alias("orders")
    )
    .orderBy(F.desc("sales")))

city_stats.show()
```

这已经非常接近 SQL 的 GROUP BY。更重要的是：groupBy(city) 要求相同 city 的记录最终能够一起聚合，因此在分区不满足要求时仍然需要 Shuffle。

DataFrame 的 groupBy 仍然可能触发 Shuffle



“DataFrame 更高级”不等于“没有 Shuffle”；只是 Spark 能更好地理解并优化这些操作。

图5 DataFrame 的 groupBy 背后仍然可能发生 Shuffle

9. DataFrame API 与 SQL 到底是什么关系

Spark 并不是维护“两套完全不同的计算引擎”。DataFrame API 和 SQL 都会进入 Spark SQL 的分析与优化体系，最后生成物理执行计划。

DataFrame API 与 Spark SQL：两种写法，最终进入同一套 Spark SQL 引擎

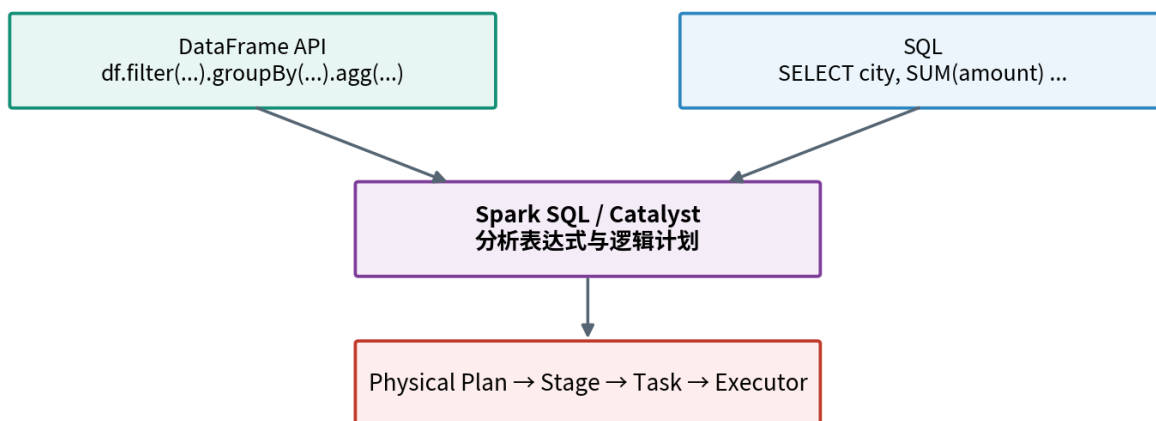


图6 DataFrame API 与 SQL 最终进入同一 Spark SQL 引擎

例如前面的城市销售统计可以写成 SQL：

```
df.createOrReplaceTempView("sales")

result = spark.sql("""
SELECT
    city,
    SUM(amount) AS sales,
    COUNT(*) AS orders
FROM sales
WHERE status = 'paid'
GROUP BY city
ORDER BY sales DESC
""")

result.show()
```

选择方式：DataFrame API 与 SQL 更多是表达风格的选择。复杂报表和分析人员常偏好 SQL；需要程序化组合条件、封装函数时 DataFrame API 很方便。二者可以在同一个程序中自由切换。

10. 什么是临时视图 Temp View

`createOrReplaceTempView("sales")` 并不是把 `DataFrame` 复制一份再创建一张真正的数据库表。可以把它理解为给当前 `SparkSession` 中的一个逻辑查询对象注册了名字，使 SQL 能通过 `FROM sales` 引用它。

误区：Temp View 通常与当前 `SparkSession` 生命周期相关，不应把它理解成永久存储。是否持久化、是否跨会话存在，是 Catalog / Hive Metastore / Table 等更进一步的话题。

11. Join：DataFrame 为什么很适合关系型分析

假设还有商品维表 products：product_id、product_name、category。我们希望给销售记录补上商品类别。

```
products = spark.read.parquet("hdfs:///dim/products")
enriched = df.join(products, on="product_id", how="left")
enriched.select(
    "order_id", "product_name", "category", "amount"
).show()
```

Join 是否发生 Shuffle、采用 SortMergeJoin 还是 BroadcastHashJoin，并不是简单由“写了 join()”这一个事实决定。数据规模、统计信息、Join 条件、配置等都会影响物理计划。第 16 讲会专门学习如何用 explain() 看见这些决策。

与 MapReduce 的连接：你在 MapReduce 中学过 Reduce-side Join 和 Map-side/Broadcast 思路。Spark SQL 的 Join 本质仍然在解决“两个分布式数据集怎样让匹配 Key 的记录相遇”，只是优化器可以自动选择更多执行策略。

12. NULL：真实数据分析中不能回避的问题

SQL / DataFrame 中的 NULL 不是普通的 0，也不是空字符串。它表示“未知/缺失”。很多 SQL 表达式遇到 NULL 会得到 NULL，而聚合函数对 NULL 的处理也有自己的规则。

```
# 查看缺失 amount 的行
df.filter(F.col("amount").isNull()).show()

# 删除关键字段缺失记录
clean = df.na.drop(subset=["product_id", "amount"])

# 对部分列填充值
filled = df.na.fill({"city": "UNKNOWN"})
```

初学者常错：不要用 col("x") == None 判断 NULL。PySpark 中使用 isNull() / isNotNull()，或者相应 SQL IS NULL / IS NOT NULL。

13. CSV、JSON、Parquet：为什么文件格式会影响效率

DataFrame 不只是一套 API，它也和结构化存储格式紧密相关。CSV 是文本格式，通用但缺少丰富类型信息；Parquet 是列式格式，通常会保存 Schema，并能让 Spark 在只读取少数列时减少不必要的数据读取。

格式	特点	适合场景
CSV	人可读、通用；类型信息弱	交换、小数据、教学、简单导入导出
JSON	半结构化、嵌套表达方便	日志、接口数据、复杂结构
Parquet	列式、Schema、压缩和分析友好	Spark/Hive 离线分析的常见选择
ORC	列式、分析型存储	Hive 生态和分析任务中常见

底层理解：如果查询只需要 city 和 amount 两列，列式格式有机会只读取相关列；这就是“列裁剪”能够真正减少 I/O 的基础之一。

14. 一个非常重要的性能原则：尽量让 Spark 看得懂你的逻辑

对结构化数据而言，性能优化的第一步通常不是手动调 JVM 参数，而是尽量使用 Spark 能理解的结构化表达式。

写法	Spark 能否理解语义	一般建议
内置 Column 函数	高	优先
Spark SQL 表达式	高	优先
Python UDF	较低	必要时使用
把大数据 collect 到 Python 再算	绕开分布式执行	大数据任务避免

原因很简单：优化器只能优化它能够“看见”的逻辑。如果你把一大段逻辑藏进任意 Python 函数，它能做的优化就会减少。

15. 上机实验：从 CSV 到 SQL 聚合

下面这个实验的目的不是追求数据量，而是让你亲手观察 Schema、DataFrame API、SQL 和执行计划之间的关系。

```
mkdir -p ~/spark_lab15
cat > ~/spark_lab15/sales.csv <<'EOF'
order_id,product_id,city,amount,status
1001,10,Beijing,299.0,paid
1002,11,Shanghai,99.0,cancelled
1003,10,Beijing,499.0,paid
1004,12,Guangzhou,199.0,paid
1005,11,Shanghai,399.0,paid
1006,12,Beijing,159.0,paid
EOF
```

pyspark

```
from pyspark.sql import functions as F

df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv("file:///home/hadoop/spark_lab15/sales.csv"))

df.printSchema()
df.show()

paid = df.filter(F.col("status") == "paid")
result = (paid.groupBy("city")
          .agg(F.sum("amount").alias("sales"),
              F.count("*").alias("orders"))
          .orderBy(F.desc("sales")))

result.show()
result.explain()
```

如果你的 Linux 用户名不是 hadoop，把 file:///home/hadoop/... 改成实际用户目录。也可以直接使用本地相对路径。

观察重点：先不要试图完全看懂 explain() 的每一行。你只需要寻找 Filter、Aggregate、Sort、Exchange 等关键词。特别是 Exchange，通常提示发生了数据重新分区/Shuffle。第 16 讲会系统拆解执行计划。

16. 再把同一任务改写成 SQL

```
df.createOrReplaceTempView("sales")

sql_result = spark.sql("""
SELECT city,
       SUM(amount) AS sales,
       COUNT(*) AS orders
FROM sales
WHERE status = 'paid'
GROUP BY city
ORDER BY sales DESC
""")

sql_result.show()
sql_result.explain()
```

然后比较两个 explain()。你会发现：不同写法常常可以得到非常接近甚至相同的逻辑/物理执行结构。这就是“API 不同、引擎相同”的直接证据。

17. 初学者最常见的 12 个疑问

- 1. DataFrame 是 Pandas DataFrame 吗？** 不是。API 外观有相似处，但 Spark DataFrame 是分布式、惰性执行的；Pandas DataFrame 通常在单机进程内。
- 2. DataFrame 是否已经完全取代 RDD？** 结构化业务分析通常优先 DataFrame/SQL，但 RDD 仍是理解 Spark 底层分区、依赖和一些底层操作的重要抽象。
- 3. DataFrame 是否一定在内存？** 不是。它描述分布式计算；数据可能来自磁盘、HDFS、对象存储，也可能在执行中 Spill。
- 4. show() 是 Transformation 吗？** 不是。show() 需要取得实际结果，因此会触发执行。
- 5. select() 会马上读完整个文件吗？** 通常不会立即执行完整 Job；它先进入计划，后续 Action 才触发执行。
- 6. groupBy() 为什么可能慢？** 相同分组 Key 需要聚在一起，通常涉及 Shuffle。
- 7. SQL 会比 DataFrame API 更慢吗？** 不能这样判断。二者都进入 Spark SQL 引擎，很多情况下可生成相同或近似计划。
- 8. inferSchema 为什么方便但不总适合生产？** 它依赖数据推断类型，可能增加扫描成本，也可能受异常样本影响。
- 9. 为什么推荐 Parquet？** 列式存储、Schema、压缩以及与 Spark SQL 优化的配合通常更适合分析型任务。
- 10. Column 为什么不能直接当 Python 值 if col > 3？** Column 是分布式表达式，不是当前 Driver 上的一个普通布尔值。
- 11. NULL 和 0 一样吗？** 完全不一样。NULL 表示未知/缺失，SQL 有专门的三值逻辑和空值语义。
- 12. 为什么不应该滥用 Python UDF？** 它会让 Spark 更难理解内部语义，并可能带来额外跨语言序列化成本；内置函数通常更容易优化。

18. 练习：把业务需求翻译成 DataFrame 操作

仍然使用 sales.csv，先独立思考，再看参考答案。

1. 统计每个 product_id 的 paid 销售额，并按销售额从高到低排序。
2. 找出 amount >= 300 的 paid 订单，只保留 order_id、product_id、amount 三列。
3. 统计每个城市的平均订单金额、最大订单金额和订单数。
4. 增加一列 level：amount >= 300 为 high，否则为 normal。
5. 解释上面哪些操作最可能引起 Shuffle，为什么。

19. 参考答案

```
# 1
(df.filter(F.col("status") == "paid")
 .groupBy("product_id")
 .agg(F.sum("amount").alias("sales"))
 .orderBy(F.desc("sales")))

# 2
(df.filter((F.col("status") == "paid") & (F.col("amount") >= 300))
 .select("order_id", "product_id", "amount"))

# 3
(df.groupBy("city")
 .agg(F.avg("amount").alias("avg_amount"),
      F.max("amount").alias("max_amount"),
      F.count("*").alias("orders")))

# 4
(df.withColumn(
  "level",
  F.when(F.col("amount") >= 300, "high").otherwise("normal")
))
```

第 1、3 题的 groupBy 通常需要按照 Key 重分区，因此常有 Shuffle；orderBy 为了形成全局顺序，也可能涉及 Shuffle。第 2、4 题如果只是局部 filter/select/withColumn，通常属于窄依赖式的逐分区处理。

20. 本讲最终心智模型

四层心智模型：① RDD 只告诉 Spark “这里有分布式对象”，DataFrame 进一步提供列名、类型和结构；② select/filter/groupBy/join 主要是在构造 Spark SQL 能理解的关系表达式与计划；③ Action 触发后，计划经过分析和优化，再变成 Stage/Task/Executor；④ DataFrame 没有消灭 Partition、Shuffle、网络、磁盘和数据倾斜，只是让 Spark 更有机会自动选择更好的执行方式。

下一讲（第 16 讲）会专门拆 Spark SQL 的“大脑”：Catalyst Optimizer。我们会从 parsed logical plan、analyzed plan、optimized logical plan 一路看到 physical plan，并用 explain() 观察 Filter 下推、列裁剪、Exchange、BroadcastHashJoin、SortMergeJoin 等真正的执行决策。